FULL-STACK PROJECT SPECIFICATION

Spring Boot Backend + ReactJS Frontend

Group Final Project | 25-Day Development Window | Spring Semester 2025–2026

1. Purpose of This Document

This specification defines the technical and functional requirements for the Full-Stack Group Project. It replaces and supersedes all previous individual project specifications. Students should use this document as the single authoritative reference for what is expected in terms of architecture, technology stack, features, and deliverables.

All groups must build a full-stack web application using Spring Boot as the backend and ReactJS as the frontend. The specific domain and use case will differ per group (assigned by the instructor), but all projects must conform to the requirements described herein.

2. Technology Stack

Layer	Technology	Version / Notes
Backend Framework	Spring Boot	3.x (latest stable)
Language	Java	Java 17 or later
Build Tool	Maven or Gradle	Either is acceptable
ORM / Data Access	Spring Data JPA + Hibernate	Configured via application. Properties
Database	PostgreSQL	v14 or later
Security	Spring Security	JWT-based authentication
Frontend Framework	ReactJS	v18.x
HTTP Client	Axios	For API consumption
Routing	React Router DOM	v6.x
Styling	CSS / Tailwind CSS	Either approach is acceptable
Version Control	Git	Repository link to be submitted

3. Application Architecture

All projects must follow a standard layered architecture. The diagram below describes the expected interaction between components:

Architecture Overview

[Browser / React App]
| HTTP Requests (REST API)

```
      v
[Spring Boot Backend]
  +----------------+
  | Controller Layer   | <-- Receives and routes HTTP requests
  +----------------+
  | Service Layer      |  <-- Business logic and rules
  +----------------+
  | Repository Layer  | <-- Data access via JPA
  +----------------+
      | SQL Queries
      v
[PostgreSQL Database]
```

4. Backend Requirements (Spring Boot)

4.1 Project Setup

Initialize the Spring Boot project using Spring Initializer (start.spring.io) with the following dependencies:

- Spring Web
- Spring Data JPA
- Spring Security
- PostgreSQL Driver
- Validation (Bean Validation / Hibernate Validator)
- Lombok (optional, for reducing boilerplate)

4.2 Application Configuration

The application. Properties (or application.yml) file must include:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/your_db_name
spring.datasource.username=your_username
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
server.port=8080
```

4.3 Entity Layer

Define JPA entities that model your application domain. Each entity must:

- Be annotated with @Entity and @Table
- Have a primary key annotated with @Id and @GeneratedValue
- Define appropriate relationships (@OneToMany, @ManyToOne, @ManyToMany) where applicable
- Use @Column annotations with constraints (nullable, unique, length) where needed
- Apply @JsonIgnore or DTOs where required to prevent serialisation issues

4.4 Repository Layer

Create Spring Data JPA repositories for each entity:

- Extend JpaRepository<Entity, ID>
- Define custom query methods using method naming conventions or @Query annotations as needed

4.5 Service Layer

All business logic must reside in the service layer:

- Annotate service classes with @Service
- Use constructor injection (preferred) or @Autowired for dependencies
- Handle exceptions gracefully and throw custom exceptions where appropriate
- Apply @Transactional on methods that involve multiple database operations

4.6 Controller Layer (REST API)

REST endpoints must follow standard HTTP conventions:

HTTP Method	Endpoint Example	Purpose
GET	/api/resource	Retrieve all records
GET	/api/resource/{id}	Retrieve a single record by ID
POST	/api/resource	Create a new record
PUT	/api/resource/{id}	Update an existing record
DELETE	/api/resource/{id}	Delete a record by ID

- Use @RestController and @RequestMapping to define API paths
- Use @RequestBody for POST/PUT request payloads
- Use @PathVariable and @RequestParam for URL parameters
- Return appropriate HTTP status codes using ResponseEntity

4.7 Authentication & Authorization

Implement secure access control using Spring Security with JWT:

- User registration endpoint: POST /api/auth/register
- Login endpoint: POST /api/auth/login — returns a JWT token on success
- Protect all non-public API endpoints with JWT authentication
- Define at minimum two roles: ROLE_USER and ROLE_ADMIN
- Restrict access to admin-only endpoints using @PreAuthorize or SecurityFilterChain configuration

4.8 Data Validation

- Apply Bean Validation annotations on request/entity fields: @NotNull, @NotBlank, @Size, @Email, @Min, @Max, etc.
- Use @Valid in controllers to trigger validation
- Return meaningful error messages in the response body when validation fails

5. Frontend Requirements (ReactJS)

5.1 Project Setup

Initialise the React application using Vite or Create React App:

```
# Using Vite (recommended)
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm install axios react-router-dom
```

5.2 Project Structure

Organise the project into a clear and logical folder structure:

```
src/
  components/       # Reusable UI components
  pages/            # Page-level components (one per route)
  services/         # API call functions (Axios)
  context/          # React Context (if used for state management)
  hooks/            # Custom React Hooks
  utils/            # Utility functions and helpers
  App.jsx           # Root component with routing
  main.jsx          # Entry point
```

5.3 Routing

- Use React Router DOM v6 to implement client-side routing
- Define routes in App.jsx using <BrowserRouter>, <Routes>, and <Route>
- Protect authenticated routes using a PrivateRoute wrapper component that checks for a valid JWT token

5.4 API Integration

- All HTTP calls to the backend must go through Axios
- Create an axios instance with a base URL and an interceptor to attach the JWT token to request headers
- Handle API errors gracefully and display user-friendly messages

```
// services/api.js
import axios from 'axios';

const api = axios.create({ baseURL: 'http://localhost:8080/api' });

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

5.5 State Management

- Use React Hooks (useState, useEffect, useContext) for component-level and global state
- For global state (e.g. authentication, user session), use React Context API
- Avoid prop drilling beyond two levels — use context or lift state appropriately

5.6 UI & Responsive Design

- All pages must be responsive and functional on desktop, tablet, and mobile viewports
- Implement a consistent navigation bar with links to all major sections
- Forms must include client-side validation with visible error messages before submission
- Display loading indicators (spinner or skeleton) while data is being fetched from the API
- Display appropriate empty-state messages when no data is returned

5.7 Authentication UI

- Registration page: form collecting at minimum username/email and password
- Login page: form submitting credentials; on success, store JWT in localStorage and redirect
- Logout: clear the token from storage and redirect to the login page
- Show/hide navigation items based on authentication state and user role

6. Note on the Use of Generative AI

⚠ Important Notice — Use of Generative AI Tools

The constructive use of Generative AI tools (e.g., ChatGPT, Claude, Gemini, Copilot) is permitted solely for educational support purposes, such as:

- Understanding programming concepts
- Learning framework features
- Debugging errors
- Exploring alternative implementation approaches
- Reviewing documentation

However, students must not use GenAI tools to generate substantial portions of the final project implementation, architecture, database design, reports, presentations, or other assessment deliverables.

The primary purpose of this assessment is to evaluate each student's ability to independently apply the knowledge and skills acquired throughout the courses.

Students must be able to explain and defend all submitted code during the project demonstration and viva session. Failure to do so may result in mark deductions or academic misconduct investigations.

7. Submission Checklist

Ensure all of the following are included in your final submission:

1. Full project source code (backend + frontend) — Git repository link
2. PostgreSQL database schema or ERD diagram
3. README file with: project overview, setup instructions, and how to run the application
4. Project report (3–5 pages): architecture, features, challenges, individual contributions
5. Acknowledgement of any GenAI tools used (included in the report)
6. Presentation slides (to be used during the Day 20 presentation)

> **Submission Deadline**
> Final Code & Report: July 15th, 23:59
> Project Defense Day will be notified.
> Late submissions will not be accepted.